EzyReadComics AI Collaboration Rules

Use these rules when helping with EzyReadComics. The repo and repo docs are the source of truth for current project state. This file defines collaboration style and delivery expectations.

Use the repo as the source of truth

When current project state matters, inspect the GitHub repo and existing docs before giving repo-specific code, architecture advice, or file replacements. Trust current files over cached memory or conversation summaries.

Useful places to check:

- README.md
- docs/development-log.md
- docs/
- catalog/models/
- catalog/views.py
- catalog/urls.py
- catalog/templates/
- catalog/management/commands/
- catalog/marvel/
- catalog/dc/
- reading/

Work incrementally

Move one step at a time. Do not jump ahead into future features unless asked. Do not add architecture, abstractions, models, services, background jobs, or cleanup just because they seem useful.

Prefer simple, direct, understandable code. When a design decision matters, explain the intended behavior and tradeoffs before giving replacement files.

Deliver complete replacement files as downloads

When changing code, templates, docs, configuration, or management commands, provide complete replacement files as downloadable artifacts.

Do not provide .patch files, .diff files, git diff output, apply_patch blocks, or line-by-line patch instructions unless the user explicitly asks for a patch or diff.

For each changed file, include:

- the repo path to replace
- a download link for the full replacement file
- a short explanation of what changed
- the exact command(s) to run next

Preferred response format:

```
Replace:
path/to/file.py

Download:
[file-name.py]

Run:
python manage.py check
```

Use one downloadable artifact per replacement file when practical. Use a zip only when the user asks for one or when there are too many files for separate links to be practical.

If artifact generation is unavailable, say so and ask whether to paste the complete file in chat. Do not fall back to a patch unless the user requested a patch.

Only paste a full file directly in the response when the user asks for pasted output, the file is small enough that pasted output is clearly better, or downloadable artifacts cannot be generated and the user agrees.

Avoid partial instructions such as:

- Add this near the top.
- Change this one block.
- Replace lines 20-40.
- Apply this patch.

This applies to Python, templates, HTML, CSS, JavaScript, README files, docs, configuration files, and management commands.

JavaScript download handling

Direct .js download links and previews can be blocked or fail in ChatGPT or the browser because JavaScript files may be treated as executable content.

For JavaScript replacement files, provide the complete file as a downloadable .js.txt artifact by default. The downloaded file should contain the full JavaScript replacement content. Tell the user to rename the file from .js.txt to .js after downloading, then replace the repo file at the stated path.

Do not provide JavaScript changes as .patch, .diff, git diff output, apply_patch blocks, or line-by-line patch instructions unless the user explicitly asks for a patch.

Use this response format for JavaScript:

```
Replace:
static/js/catalog/browse.js

Download:
browse.js.txt

After downloading, rename browse.js.txt to browse.js and replace the repo file above.
```

Use separate .js.txt downloads for separate JavaScript files when practical. Use a zip for JavaScript only when the user asks for a zip or when there are too many files for separate .js.txt downloads.

Do not over-modify

Change what was requested. Do not silently restructure unrelated code. Do not rename things unless the rename is part of the request or clearly necessary. Mention optional cleanup separately instead of doing it automatically.

Explain behavior clearly

Before replacement files, explain what the change is meant to do. For import and sync commands, clarify:

- what endpoint or data source is used
- what field, date range, page range, or URL set is scanned
- what gets created
- what gets updated
- what gets skipped
- what prevents overwrites
- what avoids unnecessary API calls, database queries, or browser reads
- whether migrations are needed

Keep explanations practical and tied to the actual code.

Keep terminal output useful

Do not add noisy command output or implementation trivia. Useful output includes:

- the command mode: dry run or apply
- the date, page, URL, or range being scanned
- progress and offsets when helpful
- created, updated, skipped, and failed counts
- whether a date, page, or window completed
- whether work was skipped because data already exists

Avoid per-item spam unless the user is actively testing and needs detailed output.

Code first, docs after it works

Normal workflow:

- Confirm intended behavior.
- Provide complete replacement file downloads.
- Give exact command(s) to run.
- Wait for the user to confirm the output works.
- Update docs after behavior is confirmed, unless the user asks to update docs immediately.
- Commit or push only when requested and available.

Preserve useful docs

When updating docs, inspect the current file and preserve useful existing content. The README should describe the current system and avoid old, unused, or deprecated implementation details unless they are needed for setup or operation.

The development log should preserve history and add new entries rather than replacing the log with a short summary. Numbered or system docs should keep useful background while removing stale instructions.

If unsure whether to overwrite or merge, provide a merged complete replacement file as a download.

Be careful with assumptions

Do not assume external API or website behavior unless verified or safely handled. When uncertain, say so and design defensively.

If the user identifies a possible flaw, re-check the reasoning before defending the previous answer. If the user is right, say so and fix it. If the user is partly right, separate the parts clearly:

- You are right about X.
- The different part is Y.
- The correct change is Z.

Match the user's pace

The user often reasons through architecture out loud. Do not interrupt with code too early. Restate the intended design, point out risks or corrections, then suggest the next concrete file or change.

Testing guidance

After code changes, give exact commands to run. Usually start with:

```
python manage.py check
```

For import and sync commands, prefer dry runs first:

```
python manage.py <command> --dry-run
```

Wait for the user to confirm output before moving to the next change. Do not assume the code worked.

Git workflow

Only move to Git after code and docs are ready. Use clear commit messages such as Add issue importer, Update Comic Vine sync docs, Fix backfill scanning, or Add volume detail filler. Avoid vague messages such as updates, stuff, fix, or changes.

GitHub access

Use GitHub access when available. Inspect files directly before giving repo-specific answers.

For this project, use the GitHub connection for reading unless the user explicitly changes that instruction. Do not claim a GitHub write succeeded unless it actually did.

If GitHub can read but not write, say so and provide complete downloadable replacement files for manual replacement. Do not provide a git patch as the fallback unless the user asks for one.

Keep scope simple

Do not add advanced comic-reading features until asked. Avoid prematurely adding:

- reading-order algorithms
- issue-to-issue connections
- event models
- character models
- creator models
- story arc models
- recommendation logic
- large background job systems
- unneeded service layers

Let the project grow gradually.

Tone

Be direct, practical, and honest. Do not over-polish. Do not use corporate-style explanations unless asked. Avoid obvious meta commentary. Focus on the next correct project move.